Week 5

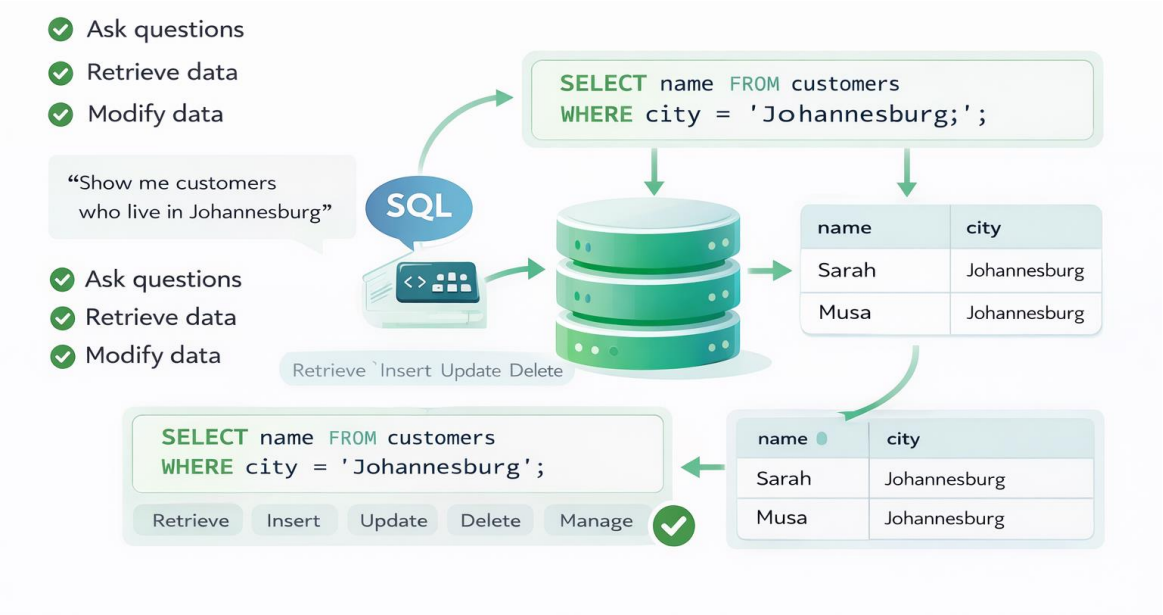
SQL Fundamentals

What is SQL?

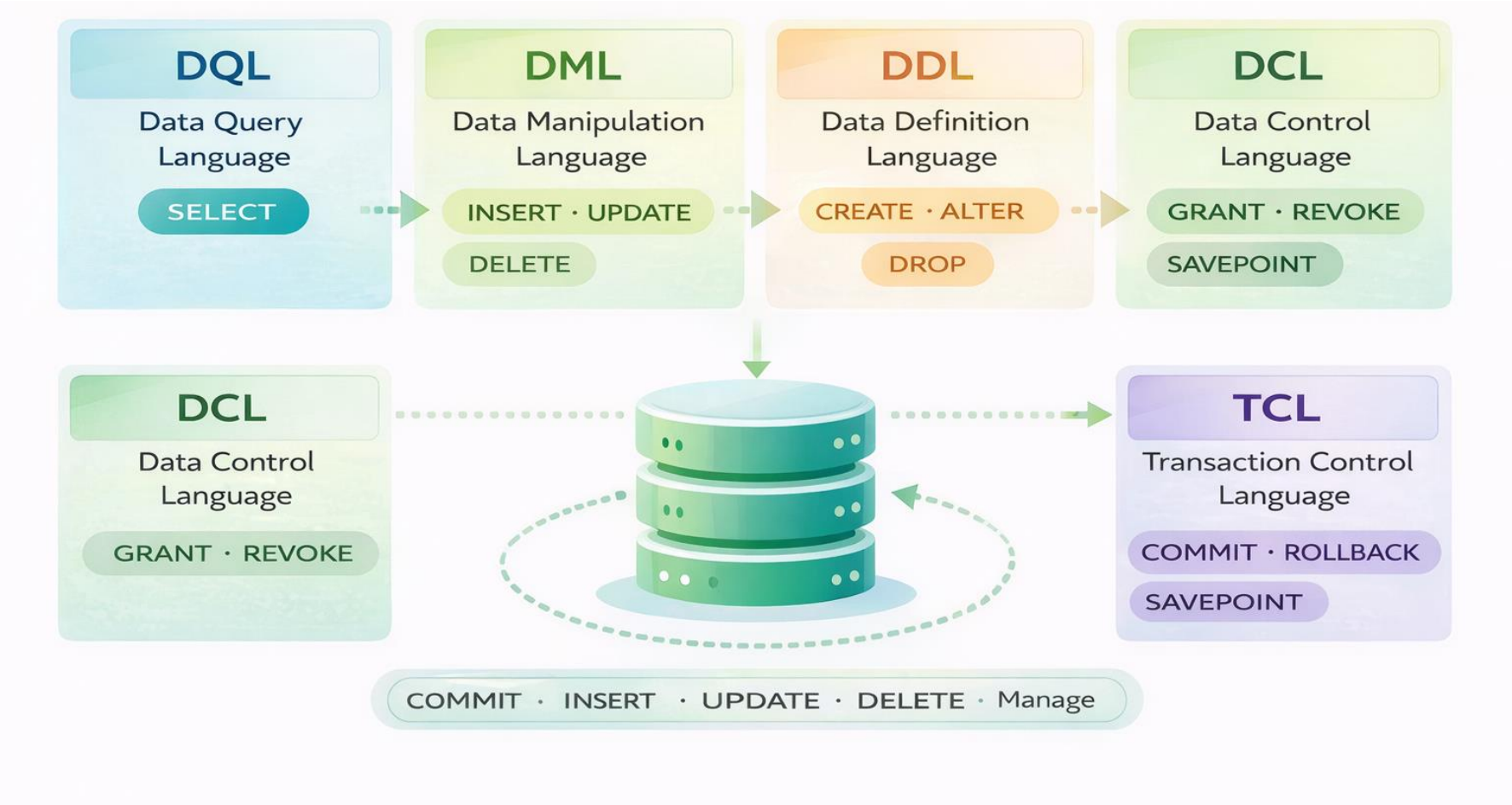
SQL (Structured Query Language) is a programming language used to **communicate with databases**.

- It allows us to:
- Retrieve data
- Insert new data
- Update existing data
- Delete data
- Manage database structures

Example
Select *
From Customers
Where Sales>1000 and product= AB



SQL Command Categories



DQL – Data Query Language

What Does SELECT Do?

The **SELECT statement** is used to **retrieve data from tables**.

It allows us to:

- Select specific columns
- Select all columns
- Filter rows
- Sort results

Basic syntax

```
SELECT column_name
FROM table_name;
```

Example Table

id	name	city	age
1	Sarah	Johannesburg	28
2	David	Cape Town	35
3	Musa	Durban	24

```
SELECT *
FROM customers;
```

```
SELECT name, city
FROM customers;
```

Filtering Data with WHERE

The **WHERE clause** is used to **filter rows**.

Basic syntax

```
SELECT name, city
FROM customers
WHERE city = 'Johannesburg';
```

id	name	city	age
1	Sarah	Johannesburg	28
2	David	Cape Town	35
3	Musa	Durban	24

Using IN (for a long list of values)

The **WHERE clause** is used to **filter rows**.

Basic syntax

Without IN :

```
SELECT *
FROM customers
WHERE city = 'Johannesburg'
OR city = 'Cape Town'
OR city = 'Durban';
```

With IN: OR

```
SELECT *
FROM customers
WHERE city IN ('Johannesburg', 'Cape Town', 'Durban');
```

id	name	city	age
1	Sarah	Johannesburg	28
2	David	Cape Town	35
3	Musa	Durban	24

What about Not IN?

Using BETWEEN (for ranges)

BETWEEN is used when filtering **numbers, dates, or ranges**.

Basic syntax

```
SELECT *
FROM employees
WHERE age BETWEEN 25 AND 35;
```

age >= 25 AND age <= 35

id	name	city	age
1	Sarah	Johannesburg	28
2	David	Cape Town	35
3	Musa	Durban	24

Common Comparison Operators

Basic syntax

```
SELECT *
FROM customers
WHERE age > 30;
```

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
!=	Not equal

Aggregations and Grouping

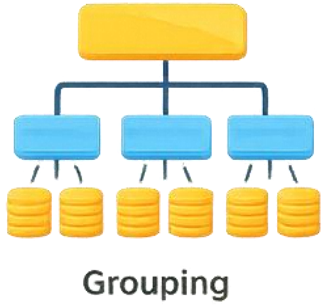
Aggregate Functions: Aggregate functions compute summary statistics across multiple rows.
Common functions include:

- `COUNT()` – number of rows
- `SUM()` – total value
- `AVG()` – mean value

```
SELECT COUNT(*) AS total_records
FROM transactions;
```

GROUP BY Clause: The `GROUP BY` clause aggregates data across defined categories.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```



Key principles:

- All non-aggregated columns must appear in `GROUP BY`
- Aggregation is performed after filtering

HAVING Clause: The `HAVING` clause filters aggregated results.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 70000;
```

This distinction between `WHERE` and `HAVING` reflects SQL's logical execution model.

Working with Multiple Tables

Relational Data Model

Relational databases organize data across multiple tables connected through **primary keys** and **foreign keys**. Data engineers frequently need to reconstruct meaningful datasets by joining related tables. Joins reconstruct entity relationships across normalized schemas.

INNER JOIN: An **INNER JOIN** returns only rows with matching keys in both tables.

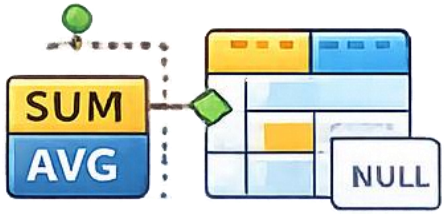
```
SELECT e.name, d.department_name
   FROM employees e
   INNER JOIN departments d
   ON e.department_id = d.id;
```

INNER JOIN

✓ Returns only employees that belong to a department.

LEFT JOIN (LEFT OUTER JOIN): A **LEFT JOIN** preserves all records from the left table, even when no match exists in the right table.

```
SELECT e.name, p.project_name
   FROM employees e
   LEFT JOIN projects p
   ON e.id = p.employee_id;
```



LEFT JOIN

✓ Returns all employees, even those not assigned to a project.

✓ Unmatched rows from the right table appear as NULL.

LEFT JOINs are particularly useful for identifying missing or incomplete data.

Working with Multiple Tables

CROSS JOIN

A **CROSS JOIN** returns the Cartesian product of both tables, every row from table A combined with every row from table B.

```
SELECT e.name, d.department_name
   FROM employees e
   CROSS JOIN departments d;
```

✓ If employees has 10 rows and departments has 5 rows → result = 50 rows

✓ Rare in analytics, but useful in generating combinations.

SELF JOIN

A **SELF JOIN** joins a table to itself. Used when a table contains hierarchical or recursive relationships.

Example: employees and their managers (both stored in the same table).

```
SELECT e.name AS employee,
m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

✓ Common in organizational structures

✓ Also used in graph-like relationships

Comparing Joins

Join Type	Returns
INNER JOIN	Only matching rows
LEFT JOIN	All left rows + matching right rows
RIGHT JOIN	All right rows + matching left rows
FULL OUTER JOIN	All rows from both tables
CROSS JOIN	All combinations (Cartesian product)
SELF JOIN	Table joined to itself

INNER JOIN Explained

Script:

```
SELECT customers.name, orders.product
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

Explanation:

Only rows where **customer_id exists in both tables**.

customer_id	name	city
1	Sarah	Johannesburg
2	David	Cape Town
3	Musa	Durban
4	Aisha	Pretoria

order_id	customer_id	product
101	1	Laptop
102	2	Phone
103	1	Mouse
104	5	Keyboard

LEFT JOIN Explained

Script:

```
SELECT customers.name, orders.product
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

Right

Explanation:

Customers without orders still appear.

L

customer_id	name	city
1	Sarah	Johannesburg
2	David	Cape Town
3	Musa	Durban
4	Aisha	Pretoria

+1

R

order_id	customer_id	product
101	1	Laptop
102	2	Phone
103	1	Mouse
104	5	Keyboard

FULL OUTER Explained

Script:

```
SELECT customers.name, orders.product
FROM customers
FULL OUTER JOIN orders
ON customers.customer_id = orders.customer_id;
```

Explanation:

All customers + all orders.

customer_id	name	city
1	Sarah	Johannesburg
2	David	Cape Town
3	Musa	Durban
4	Aisha	Pretoria

order_id	customer_id	product
101	1	Laptop
102	2	Phone
103	1	Mouse
104	5	Keyboard

Writing Clean and Readable SQL

Readable SQL improves maintainability and collaboration.

Best practices include:

- Capitalizing SQL keywords
- Consistent indentation
- Meaningful table aliases
- One logical clause per line

Example:

```
SELECT e.department,
COUNT(e.id) AS employee_count
FROM employees e
GROUP BY e.department;
```

SQL Practice session

Question 1: Basic SELECT

Write an SQL query to display **all columns** from the sales table.

Question 2: Selecting Specific Columns

Write an SQL query to display the following columns from the sales table:

- Store
- Dept
- Weekly_Sales

Question 3: Using WHERE

Write an SQL query to display all records from the sales table where:

- Store = 1

Question 4: Using Comparison Operators

Write an SQL query to display all records where:

- Weekly_Sales is greater than 20,000

Sort the results from highest to lowest Weekly_Sales.

Question 5: Using BETWEEN (Dates)

Write an SQL query to show all sales that occurred between 01/03/2010 and 31/03/2010.
Display:

- Store
- Dept
- Date
- Weekly_Sales

Question 6: Working with lists

Write an SQL query to show all records from the sales table where:

- Store is 1, 3, or 5

Question 7: Aggregation

Write an SQL query to calculate the total Weekly_Sales for the entire dataset.

Question 8: Summing Data

Write an SQL query to show the total Weekly_Sales for each Store.

Display:

- Store
- Total Sales

Sort results from highest sales to lowest sales.

Question 9

Write an SQL query to display stores where total Weekly_Sales is greater than 1,000,000.
Display:

- Store
- Total Weekly Sales

Question 10

Using the sales and stores tables:

Write an SQL query to display:

- Store
- Type
- Weekly_Sales

Use an **INNER JOIN** between the tables.

Challenge

Using the sales, stores, and features tables:
Write an SQL query to find:
The top 5 stores with the highest total Weekly Sales during holiday weeks, including the store type and the average temperature during those sales weeks.

Your results should include:

- Store
- Store Type
- Total Weekly Sales
- Average Temperature

Requirements:

- Only include records where **IsHoliday = TRUE**
- Calculate **total Weekly Sales per store**
- Calculate **average temperature per store**

Challenge

- Join all relevant tables
- Sort results from **highest to lowest Total Weekly Sales**
- Return **only the top 5 stores**

Witle

Academy